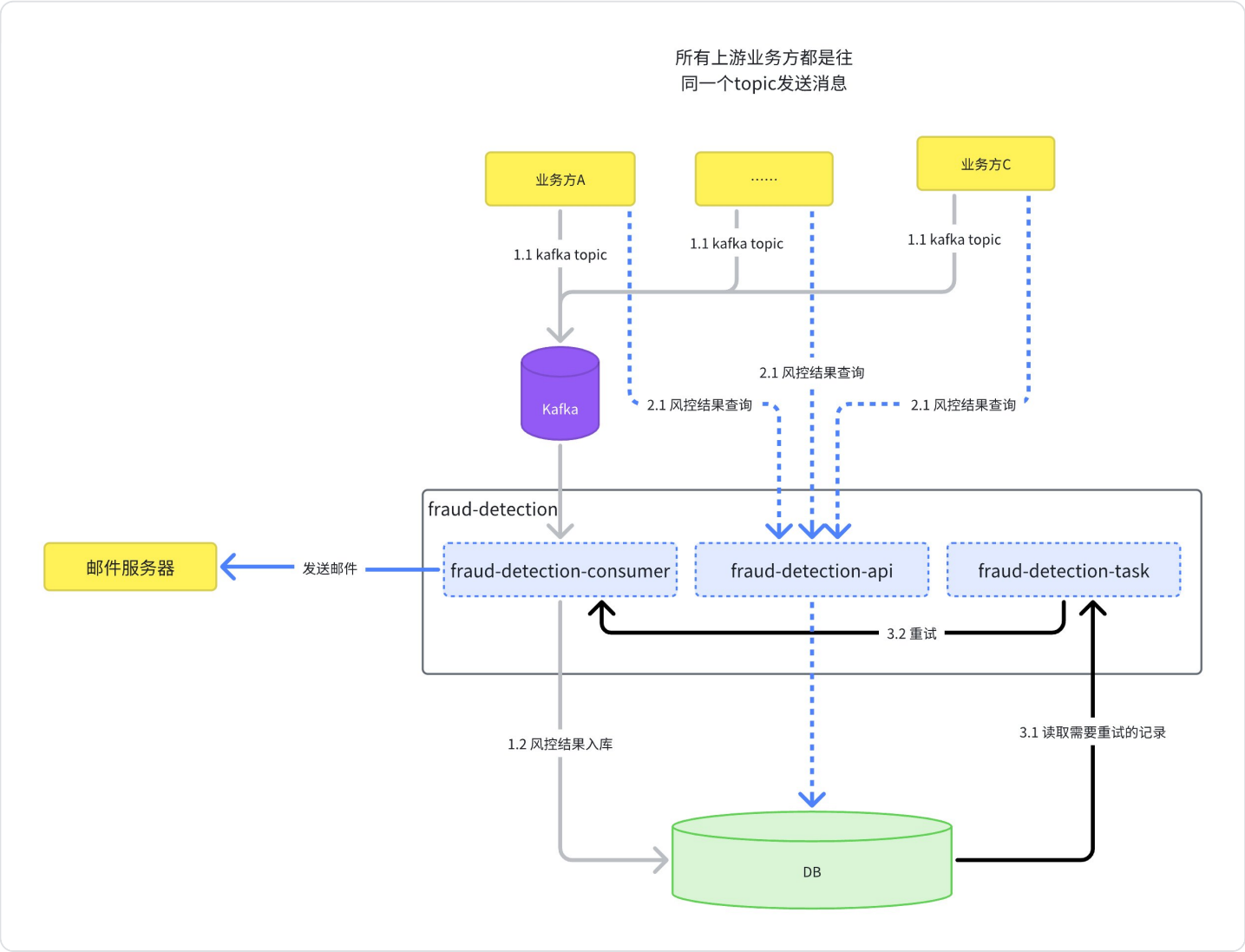


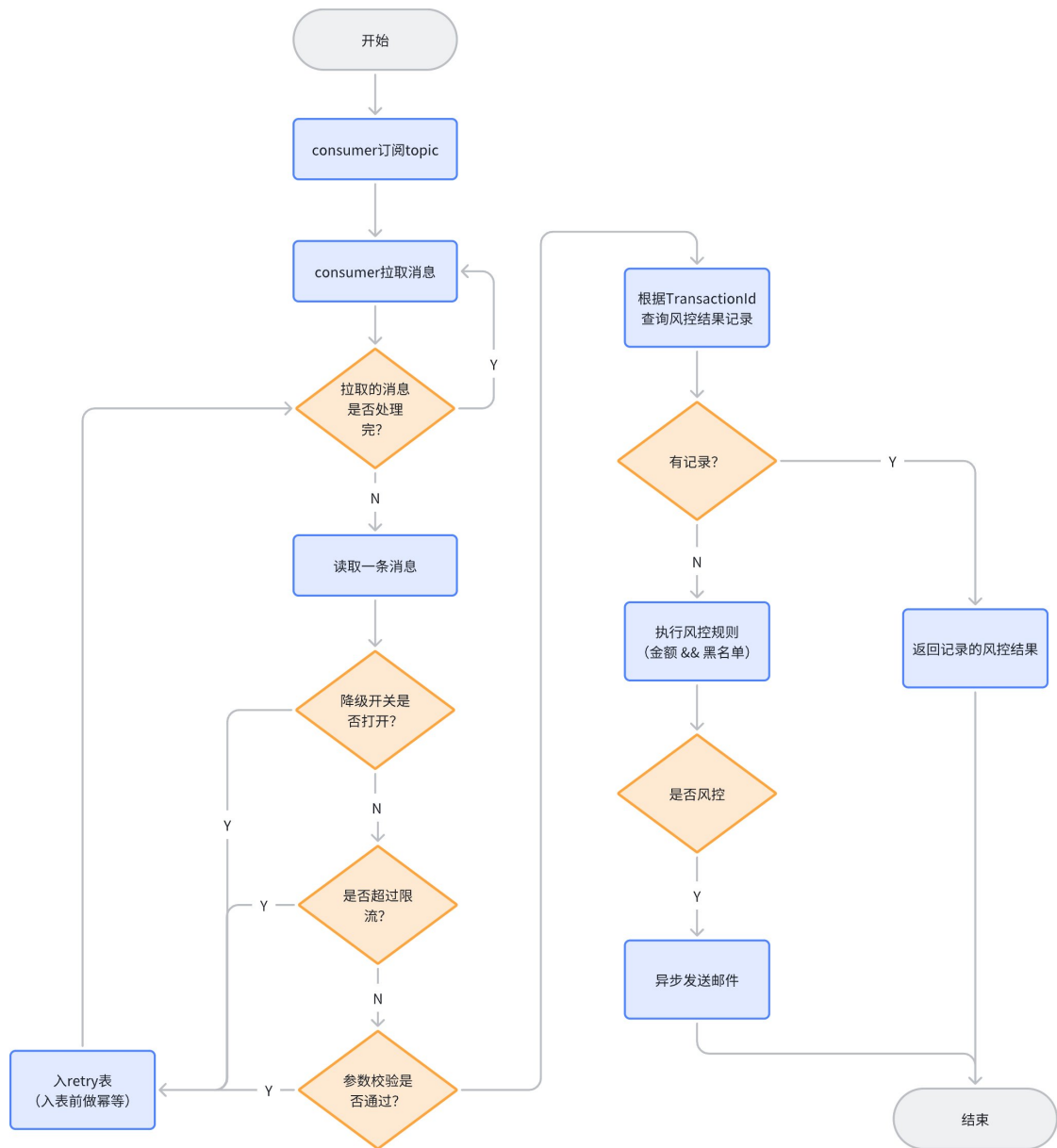
HSBC homework

1. 当前版本

1.1 架构设计：

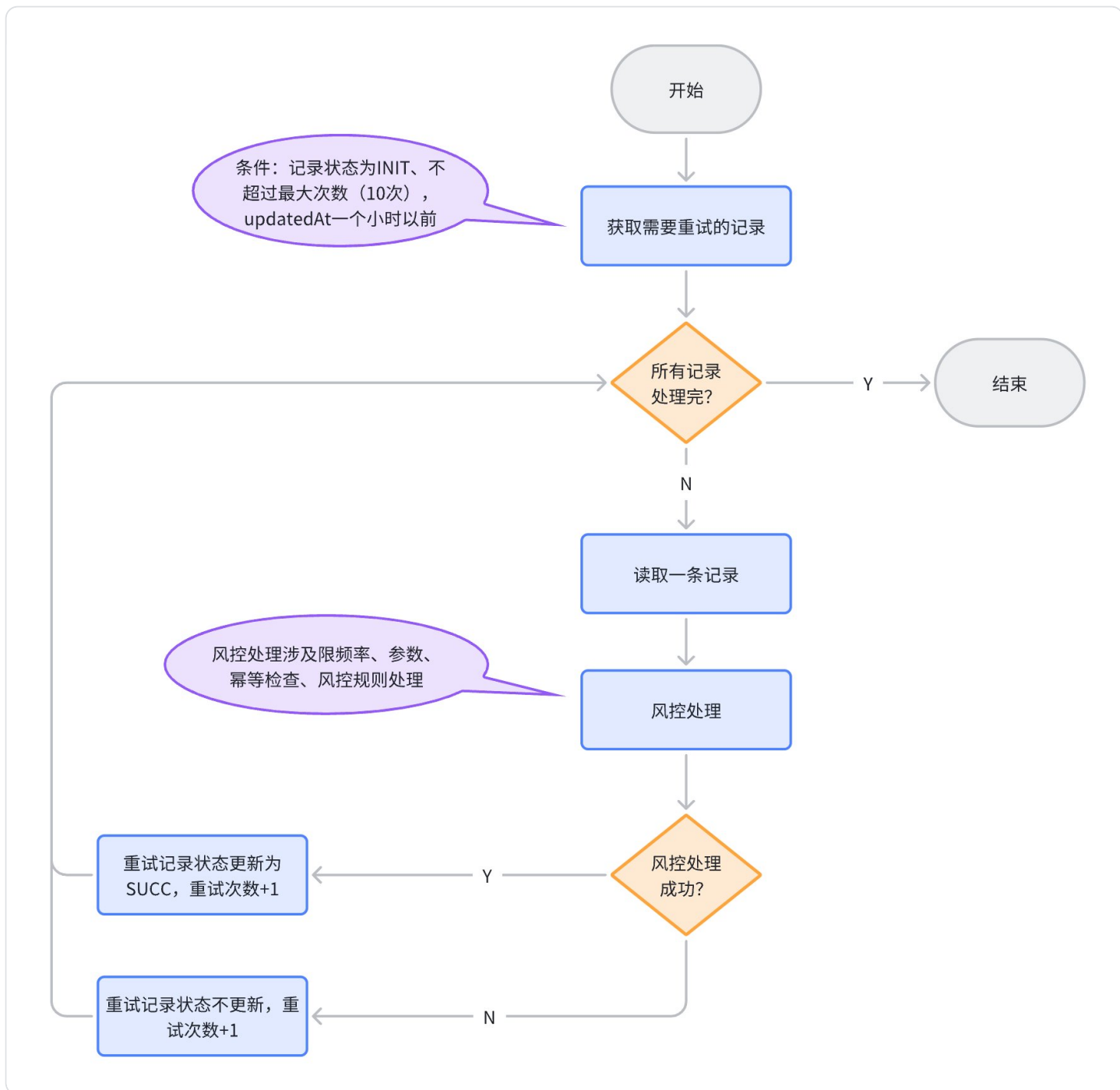


1.2 程序主链路逻辑



- 风控流程中出现异常时记录也会入retry表
- 参数错误的异常在retry表有标识，不会重试

1.3 程序定时重试链路逻辑

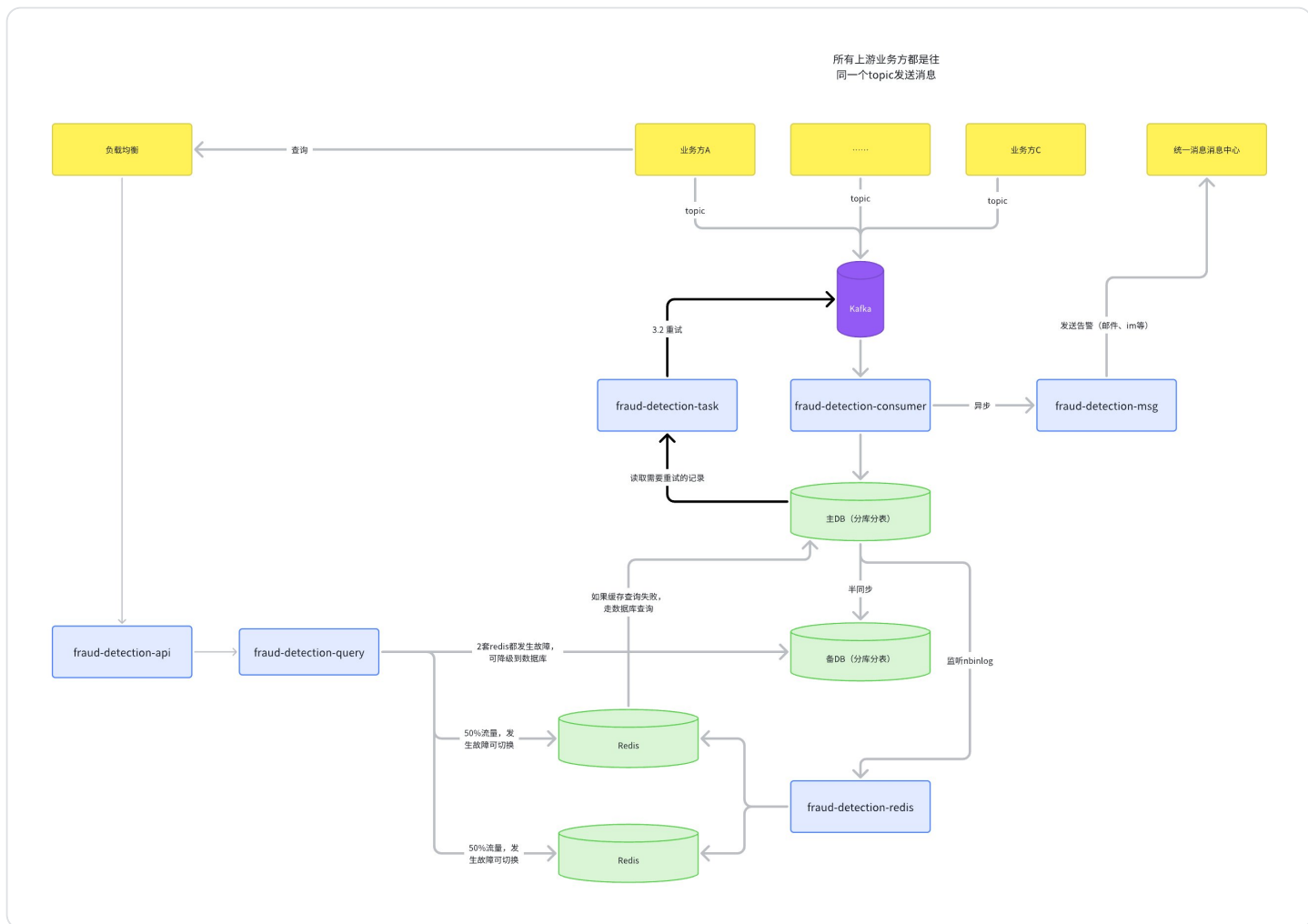


1.4 可以优化的点：

1. 消息处理时调业务方接口查询，确保消息是由上游系统发起；
2. 查询接口增加一个布隆过滤器，减少不存在记录的查询；
3. 如果风控记录到一定量级，考虑分库分表；
4. 系统压测，根据压测结果做好全局限流，选择更好的限流组件；
5. 优化日志，检查日志输出的必要性；

6. 告警邮件目前是直接发送，发送失败无感知。可以考虑邮件通知消息放Kafak，由Kafka的重试机制和死信队列保证尽量发送成功；如果邮件服务器一直无法发送，考虑降级或熔断；
7. task增加以下功能：
 - a. 定时统计达到最大发送次数仍然未成功的记录，发送告警；
 - b. 定时统计一段时间内被风控记录的总数，发送告警；
8. 增加监控：
 - a. 资源监控：CPU、内存、磁盘、网络等
 - b. Kafka 监控：发送耗时、消费速度、消息堆积；
 - c. 数据库：总连接数、空闲连接数、活跃连接数、慢查询、大事务；
 - d. 服务本身：QPS、耗时、成功率；
 - e. 异常监控：
 - f. 业务趋势监控；
9. 增加和业务方的核对
10. 现在风控规则简单，如果后续规则很多，可以考虑存入数据库。在程序启动时把规则加载到本地缓存，同时程序里面写一个定时任务更新本地缓存；
11. 如果后续业务逻辑越来越复杂，做好资源隔离：例如线程池的隔离；

2. 架构可演进的方向



3. 当前版本测试:

3.1 单侧覆盖率

根据不同的保障等级，设定程序单侧覆盖率，达到xx覆盖率才能发版。

目前该程序单侧覆盖率：91%

Coverage frauddetection in fraud-detection ✕			
Element ^	Class, %	Method, %	Line, %
com.example.frauddetection controller dto Entity Enum exception kafka repository service util FraudDetectionApplication	88% (15/17) 100% (1/1) 100% (1/1) 100% (2/2) 100% (2/2) 100% (1/1) 100% (4/4) 100% (0/0) 100% (2/2) 66% (2/3) 0% (0/1)	89% (59/66) 100% (3/3) 88% (8/9) 72% (13/18) 100% (6/6) 100% (1/1) 100% (16/16) 100% (0/0) 100% (8/8) 100% (4/4) 0% (0/1)	91% (210/230) 73% (14/19) 90% (10/11) 72% (13/18) 100% (14/14) 100% (1/1) 100% (69/69) 100% (0/0) 93% (72/77) 85% (17/20) 0% (0/1)

3.2 所有单侧需要都跑通过

The screenshot displays the IntelliJ IDEA interface during a test run. The main window shows a tree view of the test results for the project 'frauddetection (com.example)'. The test 'testConsumeMsg_demotionSwitchOn' is marked as failed (red X), while the others are passed (green checkmarks). The total execution time is 48 sec 853 ms.

Test Name	Result	Duration
frauddetection (com.example)	Failed	48 sec 853 ms
EmailNotifyTest	Passed	1 sec 914 ms
test	Passed	1 sec 914 ms
MessageConsumerTest2	Failed	5 sec 437 ms
testConsumeMsg_demotionSwitchOn	Failed	5 sec 437 ms
MessageConsumerTest	Passed	34 sec 180 ms
testConsumeMsg_idempotentPass	Passed	5 sec 94 ms
testConsumeMsg_inputError_checkHasRecord	Passed	5 sec 16 ms
testConsumeMsg_normal_pass	Passed	5 sec 11 ms
testConsumeMsg_rate_limiter_checkHasRecord	Passed	2 sec 10 ms

The right sidebar shows the test output log, which includes the following information:

- Tests failed: 1, passed: 17 of 18 tests – 48 sec 853 ms
- Library/Java/JavaVirtualMachines/jdk1.8.0_23
- 22:34:45,666 |-INFO in ch.qos.logback.classic [logback-test.xml]
- 22:34:45,670 |-INFO in ch.qos.logback.classic [logback.groovy]
- 22:34:45,670 |-INFO in ch.qos.logback.classic [file:/Users/sventao/Documents/GK_CODE/frau
- 22:34:45,766 |-INFO in ch.qos.logback.classic set

单侧由于相互影响，全部跑会有失败用例。对跑失败可以单独执行，都是能过的。

3.3 单侧测试用例

原则是覆盖所有场景，目前覆盖已下场景：

3.3.1 场景：交易正常，不风控

com.example.fraudetection.MessageConsumerTest#testConsumeMsg_normal_pass

测试过程：通过向topic发送消息，程序处理

期望结果：fraud_detect有记录，fraud_detect状态是PASS；fraud_detect_retry无记录

测试结果：符合预期；

3.3.2 场景：交易超过限额（限额: 100000），风控

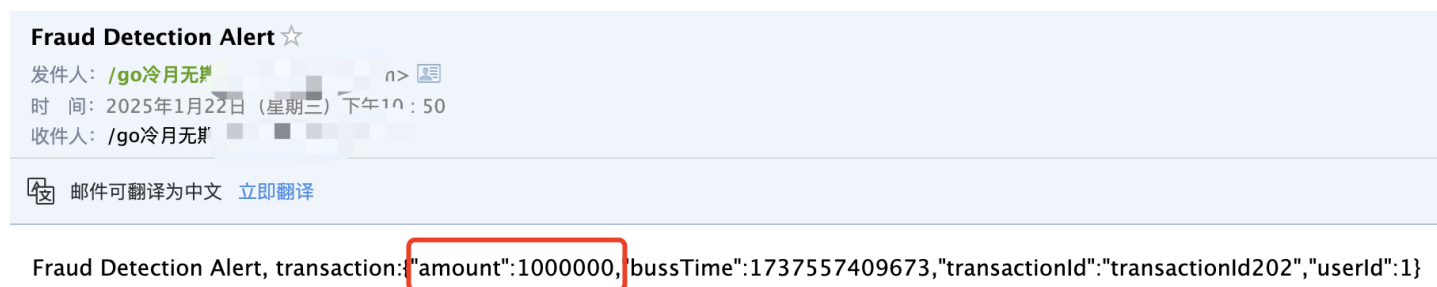
com.example.fraudetection.MessageConsumerTest#testDetectFraud_WhenAmountIsHigh

测试过程：向topic发送消息，程序处理

期望结果：fraud_detect有记录，fraud_detect状态是REJECT；fraud_detect_retry无记录；收到告警邮件

测试结果：符合预期；

告警邮件：



3.3.3 场景：交易用户在黑名单（黑名单: 123,456），风控

com.example.fraudetection.MessageConsumerTest#testDetectFraud_UserBlackListIn

测试过程：向topic发送消息，程序处理

期望结果：fraud_detect有记录，fraud_detect状态是REJECT；fraud_detect_retry无记录，收到告警邮件

测试结果：符合预期；

告警邮件：

Fraud Detection Alert ☆

发件人: /go冷月无
时 间: 2025年1月22日 (星期三) 下午10:56
收件人: /go冷月无

邮件可翻译为中文 [立即翻译](#)

Fraud Detection Alert, transaction:{"amount":100,"bussTime":1737557764450,"transactionId":"transactionId203","userId":123}

3.3.4 场景：超过限流，未限流部分入fraud_detect，被限流的入fraud_detect_retry
com.example.frauddetection.MessageConsumerTest#testConsumeMsg_rateLimiter_checkHasRecord

测试过程：向topic快速发送6条消息，程序处理

限制条件：需要com.example.frauddetection.service.FraudDetectionRetryService#doRetryScheduled 注释掉，份否则有可能被doRetry处理，导致状态不对；

期望结果：fraud_detect_retry无记录，且状态都是INIT

测试结果：符合预期；

3.3.5 场景：超过限流，校验异常类型

com.example.frauddetection.FraudDetectionConsumerTest#testConsumeMsg_rateLimiter_checkException

测试过程：快速调用com.example.frauddetection.kafka.FraudDetectionConsumer#consumeMsg接口多次，程序处理

期望结果：返回异常，且异常信息是：too busy error

测试结果：符合预期；

3.3.6 场景：消息格式错误，入fraud_detect_retry

com.example.frauddetection.MessageConsumerTest#testConsumeMsg_inputError_checkHasRecord

测试过程：向topic快速发送1条格式错误的消息，程序处理

期望结果：fraud_detect无记录；fraud_detect_retry有记录，且retry_stat == NO_RETRY && retry_times == 0

测试结果：符合预期；

3.3.7 场景：消息格式错误，校验异常类型

`com.example.frauddetection.FraudDetectionConsumerTest#testConsumeMsg_inputError_checkException`

测试过程：调用`com.example.frauddetection.kafka.FraudDetectionConsumer#consumeMsg`接口，但入参数据格式错误的消息，程序处理

期望结果：返回异常，且异常信息是：input parameter error

测试结果：符合预期；

3.3.8 场景：上游重复发送消息，幂等，直接返回数据库中风控结果（PASS）

`com.example.frauddetection.MessageConsumerTest#testConsumeMsg_idempotentPass`

前置条件：`fraud_detect`有一条PASS记录；

测试过程：向topic发送消息，消息内容是数据库已经存在的记录，程序处理

期望结果：消息发送前后该条记录的Stat 和 updatedAt 不变（如果有操作数据库，updatedAt是要更新的）

测试结果：符合预期；

3.3.9 场景：上游重复发送消息，幂等，直接返回数据库中风控结果（REJECT）

`com.example.frauddetection.MessageConsumerTest#testConsumeMsg_idempotentReject`

前置条件：`fraud_detect`有一条REJECT记录；

测试过程：向topic发送消息，消息内容是数据库已经存在的记录，程序处理

期望结果：消息发送前后该条记录的Stat 和 updatedAt 不变（如果有操作数据库，updatedAt是要更新的）

测试结果：符合预期；

3.3.10 场景：降级开关打开，记录入retry表

`com.example.frauddetection.MessageConsumerTest2#testConsumeMsg_demotionSwitchOn`

前置条件：`spring.fraud.detect.demotion.switch = true`；

`com.example.frauddetection.service.FraudDetectionRetryService#doRetry Scheduled` 注释掉，份否则有可能被doRetry处理，导致状态不对；

测试过程：向topic发送消息，程序处理

期望结果：fraud_detect无记录，fraud_detect_retry有记录，retry_stat == INIT && retry_times == 0

测试结果：符合预期；

3.3.11 场景：重试记录未达最大次数，发起重试

com.example.fraudddetection.FraudDetectionRetryServiceTest#testDoRetry_normal

前置条件：fraud_detect_retry 预先插入一条记录，retryStat == INIT && retry_times == 0

测试过程：执行com.example.fraudddetection.service.FraudDetectionRetryService#doRetry

期望结果：fraud_detect有记录，state == PASS；fraud_detect_retry记录retry_stat == SUCCESS && retry_times == 1

测试结果：符合预期；

3.3.12 场景：重试记录已达最大次数，不发起重试

com.example.fraudddetection.FraudDetectionRetryServiceTest#testDoRetry_exceedMax

前置条件：fraud_detect_retry 预先插入一条记录，retryStat == INIT && retry_times == 10

测试过程：执行com.example.fraudddetection.service.FraudDetectionRetryService#doRetry

期望结果：fraud_detect无记录；fraud_detect_retry记录retry_stat == INIT && retry_times == 10

测试结果：符合预期；

3.3.13 场景：消息格式错误，不发起重试

com.example.fraudddetection.FraudDetectionRetryServiceTest#testDoRetry_paraError_noRetry

前置条件：fraud_detect_retry 预先插入一条格式错误的记录，retryStat == NO_RETRY && retry_times == 0

测试过程：执行com.example.fraudddetection.service.FraudDetectionRetryService#doRetry

期望结果：fraud_detect无记录；fraud_detect_retry记录retry_stat == INIT && retry_times == 0

测试结果：符合预期；

3.3.14 场景：查询交易的风控结果，有记录

com.example.fraudddetection.FraudDetectionControllerTest#testController_query_hasRecord

前置条件: fraud_detect 预先插入一条PASS的记录

测试过程: 调用FraudDetectionController.query接口

期望结果: 返回的信息: transactionId: transactionId402 pass

测试结果: 符合预期;

3.3.15 场景: 查询交易的风控结果, 无记录

com.example.frauddetection.FraudDetectionControllerTest#testController_query_noRecord

前置条件: fraud_detect 不存在 transactionId403 的记录

测试过程: 调用FraudDetectionController.query接口

期望结果: 返回的信息: transactionId: transactionId403 not exist

测试结果: 符合预期;